Unsafe Rust

All those loose ends

- `[T]::split_at_mut`
- Manually allocating memory
- Handling uninitialized memory
- `Vec<T>` growing

Memory Safety

- Rust is a memory safe language (unlike C and C++)
- Memory safety means no Undefined Behavior (UB)
- “Rust is safe because all code does well-defined things”
- Why would something be undefined behavior?

Undefined Behavior

```
1  void foo(int* arr) {
2      arr[7] = 42;
3  }
4
5  int main() {
6      int numbers[] = {1,2,3,4};
7      float PI = 3.14159;
8      foo(numbers);
9      std::cout << PI << std::endl;
10 }
```

What does this program print?

```
5.88545e-44
```

Accesses past the end of an array are undefined behavior



Why not simply define what happens on an out-of-bounds memory access?

Reasons for Undefined Behavior

- Performance & compiler optimizations
- Defined behavior has to be enforced and therefore be detected

`arr[7] = 42` with UB:

```
1  ldr     r3, [fp, #-8] @ arr addr.
2  add     r3, #7        @ operator[]
3  mov     r2, #42
4  str     r2, [r3]
```

`arr[7] = 42` without UB:

```
1  ldr     r4, [fp, #-12] @ size of arr
2  cmp     r4, #7         @ size <= 7?
3  ble     .RaiseSIGSEGV
4  ldr     r3, [fp, #-8]  @ arr addr.
5  add     r3, #7         @ operator[]
6  mov     r2, #42
7  str     r2, [r3]
```

Interaction with the language

- Not every behavior can be determined at compile-time
- Performance vs. safety tradeoff!
- `operator[]` in Rust requires bounds checking
- C++ does not, so is Rust slower than C++?
- What if we really don't want (or need) the bounds check?
- Enter unsafe Rust

`unsafe` Rust

```rust
// On the slice type [T]:
pub unsafe fn get_unchecked(&self, index: usize) -> &T { ... }
```

- We can turn off the Rust safety guarantees using the `unsafe` keyword

```rust
let arr = [1,2,3,4];
// SAFETY: Index is within bounds
let val = unsafe {
    arr.get_unchecked(3)
};
println!("{val}");
```

- To call an `unsafe` function, we need an `unsafe {}` block
- Always accompany `unsafe {}` blocks with a `//SAFETY` comment!

`unsafe` is dangerous

- Each `unsafe fn` will have safety guarantees: Rules that you have to follow in order to make it safe to use this function
- These rules are not enforced by the compiler! This is the whole point of `unsafe`
- Safety guarantees are part of the method documentation:

```rust
/// # Safety
///
/// Calling this method with an out-of-bounds index is *[undefined behavior]*
/// even if the resulting reference is not used.
/// [...]
pub unsafe fn get_unchecked(&self, index: usize) -> &T { ... }
```

When is something `unsafe`? (1/4)

Calling a C function:

```
1 extern "C" {
2     fn strcpy(dest: *mut u8, src: *const u8);
3 }
```

When is something `unsafe`? (2/4)

Accessing a non-existing element in a `HashMap`:

```
let map = HashMap::<&str, i32>::default();
map.get("key");
```

When is something `unsafe`? (3/4)

Allocating a block of memory on the heap:

```
1  let layout = Layout::new::<u32>();
2  let ptr = alloc(layout);
```

When is something `unsafe`? (4/4)

Accessing a mutable global variable:

```
1  static mut COUNTER: i32 = 0;
2
3  *COUNTER += 1;
```

What we can do in `unsafe` code

- Dereference raw pointers
- Call unsafe functions (including C functions, compiler intrinsics, and the raw allocator)
- Implement unsafe traits
- Access or modify mutable statics
- Access fields of unions

Pointers

- Rust has pointers with similar syntax to C/C++:
- `*const T` and `*mut T`
- Rust pointers have no ownership and lifetime information
- Documented in **`std::ptr` module**
- Pointer usage is only safe if a pointer is valid for a given access
- But: `The precise rules for validity are not determined yet`

Using pointers

- You can always go from a Rust reference / borrow to a pointer:

```
let val = 42;
let borrow = &val;
let as_ptr = borrow as *const i32;
```

- The opposite direction also works, but is unsafe:

```
let val = 42;
let borrow = &val;
let as_ptr = borrow as *const i32;
// SAFETY: The pointer comes from a reference
let borrow_from_ptr = unsafe {
    &*as_ptr // * dereferences the pointer, & makes a borrow
};
```

Pointer to reference conversion

- Why is a pointer-to-reference conversion unsafe?
- Because Rust references (borrows) have very strict rules:
- Aliasing: The Rule Of One
- Validity: Not null, initialized, respecting the valid values (e.g. either `0` or `1` for a `bool`, nothing else)
- Alignment: The memory address is aligned to the requirement of the type
- Very easy to get wrong…

Pointer reads and writes

- Three options:

1. Dereference using `*`
2. `ptr::read`
3. `ptr::write`

- Slightly different:

```
let boxed = Box::new(42);
let ptr = &boxed as *const Box<i32>;
unsafe {
    // let box2 = *ptr; // Does not compile because Box is not Copy
    let box2 = ptr.read(); // Works, but breaks ownership
    let box3 = ptr.read_unaligned(); // Unnecessary, ptr is properly aligned
}
```

Who cares?

Let’s implement `Vec<T>` :)

`Vec<T>` v1

```
1 struct Vec<T> {
2     ptr: Box<[T]>,
3     size: usize,
4     capacity: usize,
5 }
```

- Problems:
- How do we obtain a `Box<[T]>`?
- All elements in a slice must be properly initialized! No room for uninitialized elements...

`Vec<T>` v2

```
struct Vec<T> {
    ptr: *mut T,
    size: usize,
    capacity: usize,
}
```

- Now everything becomes `unsafe`:
- Allocation
- Insertion
- Growing
- Accessing elements
- But we can wrap everything in safe functions if we are careful!

Pushing an element

Pushing an element (code)

```rust
fn push_first(&mut self, element: T) {
    const INITIAL_CAPACITY: usize = 4;
    let layout = Layout::array::<T>(INITIAL_CAPACITY)
        .expect("Invalid memory layout");
    // SAFETY: We check for null afterwards
    let arr: *mut T = unsafe { std::alloc::alloc(layout) as *mut T };
    // alloc is allowed to return a null-pointer if there is not enough memory
    if arr.is_null() {
        panic!("Out of memory");
    }
    let first = arr.add(0); // For clarity
    // SAFETY: Layout matches `T`
    unsafe {
        first.write(element); // This 'forgets' the old value
    }
    self.length += 1;
    self.ptr = arr;
    self.capacity = INITIAL_CAPACITY;
}
```

Accessing an element

```rust
fn get(&self, index: usize) -> &T {
    if index >= self.length {
        panic!("Index out of bounds");
    }
    // SAFETY: Index is within bounds and `ptr` is not null
    unsafe {
        &*self.ptr.add(index)
    }
}
```

Cleanup

- Implementing `Drop` for `Vec<T>` is important
- Deallocate the memory on the heap
- Call the destructor of every element in the `Vec<T>` (how?)

Cleanup (code)

```rust
impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        if self.ptr.is_null() {
            return;
        }

        for idx in 0..self.length {
            // SAFETY: Element is properly initialized
            let element = unsafe { self.ptr.add(index).read() };
            drop(element);
        }

        let layout = Layout::array::<T>(self.capacity)
            .expect("Invalid Layout");
        // SAFETY: ptr is not null and the Layout matches
        unsafe {
            dealloc(self.ptr as *const u8, layout);
        }
    }
}
```

Cleanup improved

```rust
impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        if self.ptr.is_null() {
            return;
        }

        // SAFETY: ptr + length elements are properly initialized
        //         and aligned
        unsafe {
            let slice: &[T] = std::slice::from_raw_parts_mut(
                self.ptr,
                self.length,
            );
            std::ptr::drop_in_place(slice.as_mut_ptr());
        }

        let layout = Layout::array::<T>(self.capacity)
            .expect("Invalid Layout");
        // SAFETY: ptr is not null and the Layout matches
        unsafe {
            dealloc(self.ptr as *const u8, layout);
        }
    }
}
```

Soundness

- The property of `unsafe` code to be safe for all possible usage patterns
- `unsafe` code can be made sound by preventing invalid usage
- e.g. bounds checks, checks for OOM
- `Vec<T>` in the standard library uses tons of `unsafe` code, but has a safe and easy-to-use API

Checking soundness using `miri`

- We can use the `miri` tool to detect UB in Rust code!
- Checks the intermediate representation of the Rust compiler output
- Requires `nightly` Rust
- Run like so: `cargo +nightly miri run`
- Live example of invalid `unsafe` code!



Speaker notes

